

[CS230] Special Applications: Neural Style Transfer (NST)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Special Applications (Current Unit)

- 10.1 Face Recognition - *Completed*
- **10.2 Neural Style Transfer**
 - * What are we optimizing? (Pixel vs Weights)
 - * Content Cost Function ($J_{content}$)
 - * Style Cost Function (J_{style}): Gram Matrix
 - * Implementation Strategy

Chapter 11. Sequence Models (RNN) - *Next Part*

지난 시간 복습 및 연결

지난 시간에 우리는 삼 네트워크를 통해 CNN이 이미지의 특징을 벡터로 압축하는 법을 배웠습니다. 그렇다면, 이 특징을 분리해서 조작할 수는 없을까요? 이미지의 '**내용(Content)**'은 내 사진인데, '**화풍(Style)**'은 반 고흐의 그림처럼 만들 수 있다면 어떨까요? 이것이 바로 AI 예술의 시초, **Neural Style Transfer(NST)**입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 CNN의 특성을 활용하여 두 이미지를 합성하는 NST 알고리즘을 다룹니다.

- **전환:** 가중치(W)가 아닌 입력 이미지(G)의 픽셀을 학습한다는 차이점을 이해합니다.
- **콘텐츠:** 깊은 층의 활성화 맵을 비교하여 내용의 유사성을 측정합니다.
- **스타일:** 그람 행렬(**Gram Matrix**)을 통해 이미지의 질감과 상관관계를 수치화합니다.
- **통합:** $J(G) = \alpha J_{content} + \beta J_{style}$ 을 최소화하여 예술 작품을 생성합니다.

2 Essential Terminology

이미지	기호	역할
Content Image	C	내용의 기준 (예: 내 사진).
Style Image	S	화풍의 기준 (예: 고희 그림).
Generated Image	G	우리가 만들 결과물 (처음엔 노이즈).
Gram Matrix	$G_{kk'}$	특성맵 채널 간의 상관관계를 나타내는 행렬.

3 Core Concepts: 무엇을 학습하는가?

3.1 1. The Big Picture

NST의 가장 큰 특징은 학습의 대상이 다르다는 것입니다.

- 기존 CNN 학습: 이미지 고정 $\rightarrow$ 가중치 W 업데이트.
- NST 학습: 가중치 W 고정 (Pre-trained) $\rightarrow$ 생성 이미지 G 의 픽셀값 업데이트.

3.2 2. Content Cost Function ($J_{content}$)

”이미지 G 가 이미지 C 와 비슷한 내용을 담고 있는가?”

- 원리: CNN의 깊은 층 (Deep Layer)은 사물의 배치나 구조 같은 고차원 정보를 담고 있습니다.
- 수식: 특정 층 l 에서 두 이미지의 활성화 맵($a^{[l]}$) 간의 차이 (MSE)를 계산합니다.

$$J_{content}(C, G) = \frac{1}{2} \|a^{[l](C)} - a^{[l](G)}\|^2$$

3.3 3. Style Cost Function (J_{style}) - [핵심]

”이미지 G 가 이미지 S 의 화풍(질감)을 담고 있는가?” 스타일은 ”어디에 있는가”가 아니라 ”무엇이 함께 나타나는가(Correlation)”입니다.

□ 그람 행렬의 직관 (직관적 비유)

어떤 층에 두 개의 필터(채널)가 있다고 가정합시다.

- 필터 A: 수직선을 찾음.
- 필터 B: 주황색을 찾음.

이 두 필터가 이미지의 같은 위치에서 동시에 활성화된다면 (상관관계 높음), ”이 화풍은 수직선과 주황색이 함께 다니는 스타일이다”라고 정의할 수 있습니다. 이것을 수치화한 것이 그람 행렬 (Gram Matrix)입니다.

□ Gram Matrix ($G^{[l]}$) (수학적 원리)

$$G_{kk'}^{[l]} = \sum_i \sum_j a_{i,j,k}^{[l]} \cdot a_{i,j,k'}^{[l]}$$

(채널 k 와 채널 k' 의 활성화 맵 내적)

스타일 비용은 두 이미지의 그람 행렬 차이입니다.

$$J_{style}^{[l]}(S, G) = \|G^{[l](S)} - G^{[l](G)}\|^2$$

4 Implementation: Optimization Loop

TensorFlow/Keras를 사용한 구현의 핵심 구조입니다.

```
1 import tensorflow as tf
2
3 # 1. 모델 준비 (VGG19, 가중치 고정)
4 vgg = tf.keras.applications.VGG19(include_top=False, weights='imagenet')
5 vgg.trainable = False
6
7 # 2. 생성 이미지(G) 초기화 학습( 대상!)
8 # 처음에는 랜덤 노이즈로 시작하거나 Content 이미지로 시작
9 generated_image = tf.Variable(content_image)
10
11 # 3. 최적화 루프
12 optimizer = tf.optimizers.Adam(learning_rate=0.02)
13
14 @tf.function
15 def train_step(image):
16     with tf.GradientTape() as tape:
17         # 모델 통과 -> 활성화 맵 추출
18         outputs = get_vgg_layers(image)
19
20         # Loss 계산
21         loss_c = calculate_content_loss(outputs, content_targets)
22         loss_s = calculate_style_loss(outputs, style_targets) # Gram Matrix 사용
23
24         total_loss = alpha * loss_c + beta * loss_s
25
26         # 핵심: 가중치가 아닌 입력 '이미지'에 대한 기울기 계산
27         grad = tape.gradient(total_loss, image)
28
29         # 이미지 픽셀 업데이트
30         optimizer.apply_gradients([(grad, image)])
31
32         # 픽셀값 클리핑 (0~1 사이 유지)
33         image.assign(tf.clip_by_value(image, 0.0, 1.0))
```

Listing 1: Neural Style Transfer Logic

5 FAQ & Pitfalls

왜 그람 행렬인가요? (오해 방지 가이드)

그람 행렬은 공간 정보(Spatial Information)를 합쳐버립니다($\sum_{i,j}$). 즉, "어디에" 있는지는 무시하고 "어떤 특징들이 통계적으로 공존하는가"만 남기기 때문에 스타일(질감, 패턴)을 표현하기에 적합합니다.

Q. α 와 β 는 어떻게 정하나요?

A. 취향입니다. α (콘텐츠)를 높이면 원본 사진과 비슷해지고, β (스타일)를 높이면 그림 화풍이 강해집니다. 보통 β 를 훨씬 크게 설정합니다(숫자 스케일 차이 때문).

다음 단계 (Next Step)

우리는 CNN을 이용해 이미지를 분류하고, 탐지하고, 심지어 예술 작품으로 변환하는 방법까지 배웠습니다. 이로써 **컴퓨터 비전(Computer Vision)** 파트를 마무리합니다.

이제 시각 정보가 아닌, **시간의 흐름이 있는 데이터(Sequence Data)**의 세계로 넘어갑니다. 다음 시간부터는 음성, 언어, 주가 등 연속적인 데이터를 처리하는 **[Part 5. Sequence Models]**를 시작하며, 그 첫 번째 주자인 **RNN (Recurrent Neural Networks)**에 대해 알아보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **NST**: 사전 학습된 CNN을 이용해 콘텐츠와 스타일을 합성한다.
2. **Learning**: 가중치는 고정하고, 입력 이미지의 픽셀을 학습(업데이트)한다.
3. **Content**: 깊은 층의 활성화 맵 차이를 줄인다.
4. **Style**: 그람 행렬(**Gram Matrix**)의 차이를 줄여 질감 상관관계를 모방한다.